

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA



Escuela Profesional de Ingenieria en Informatica y Sistemas

Diseño de sistemas

Gestión de citas de veterinaria (VetCare)

Presentado por:

- | | |
|---------------------------------|-------------|
| - Luis David Cruz Llica | 2024-119060 |
| - Jose Antonio Vilcanqui Chambi | 2024-119017 |
| - Cristian Chura Peralta | 2017-119049 |

TACNA-2026

1. Resumen ejecutivo.....	3
2. Especificación Lógica y Arquitectónica.....	4
2.1 Nivel 1 – Diagrama de Contexto (C4-L1).....	4
2.2 Nivel 2 – Diagrama de Contenedores (C4-L2).....	4
2.3 Nivel 3 – Diagrama de Componentes (C4-L3).....	5
2.4 Diagrama de Clases.....	6
3. Contratos de Componentes (OpenAPI v3.0.3).....	7
4. Plan de Aseguramiento de Calidad (SQAP).....	13
4.1 Mapeo de RNF a ISO/IEC 25010.....	13
4.2 Estrategia de Aseguramiento (Walkthrough).....	14
4.3 Casos de Prueba.....	16
5. Modelado de Seguridad (Registro STRIDE).....	18
5.1 Fronteras de Confianza sobre el C4 L2.....	18
5.2 Registro de Amenazas y Mitigación.....	19
6. Medición de Atributos Internos (Métricas CK).....	22
6.1 Suite de Métricas por Clase.....	22
6.2 Identificación de Hotspots.....	24
6.3 Propuesta de Refactorización: AppointmentFacade.....	25
7. Registro de Deuda Técnica y Plan de Remediación.....	26
8. Matriz de Trazabilidad Total y Esqueleto de Código.....	29
8.1 Matriz de Trazabilidad Total.....	29
8.2 Esqueleto de Código.....	31
9. Continuidad Unidad 1 ↔ Unidad 2 y Tesis Crítica.....	35
9.1 Decisión de U1 auditada.....	35
9.2 Cadena causal: decisión → métrica CK → amenaza STRIDE.....	35
9.3 Tesis crítica verificable.....	36
9.4 Red Teaming escrito: Contraargumento y refutación.....	36
10. Bitácora de Co-evaluación e Iteración.....	38
10.1 Hallazgo #4 – Contrato OpenAPI: endpoint DELETE /citas/{citaId}.....	38
10.2 Hallazgo #3 – Contrato OpenAPI: identificadores de negocio.....	39
10.3 Hallazgos pendientes (transparencia de iteración).....	39
10.4 Hallazgo H-01 – Contrato OpenAPI: respuesta 401 ausente.....	40
10.5 Hallazgo H-02 – Contrato OpenAPI: asimetría GET/DELETE.....	41
10.6 Aprendizaje del ciclo.....	42
11. Referencias y Anexos.....	42

1. Resumen ejecutivo

Este trabajo presenta el diseño de VetCare, un sistema pensado para que una clínica veterinaria pueda manejar todo desde un solo lugar: las mascotas registradas, las citas, las atenciones médicas, los pagos y los avisos que se le mandan al cliente.

Para plantear la arquitectura usamos el modelo C4, viéndolo desde sus tres niveles: contexto, contenedores y componentes. Ahí definimos quiénes interactúan con el sistema (el Cliente, la Recepción y el Veterinario) y con qué sistemas externos se conecta (la pasarela de pagos y el servicio de notificaciones). Por dentro, el backend quedó dividido en cinco módulos —Citas, Mascotas, Historial Clínico, Notificaciones y Pagos—, cada uno con su contrato en OpenAPI v3.0.3, y todo guardado en una base de datos MySQL en Google Cloud SQL.

Al flujo más delicado, que es crear una cita y cobrar por ella, le hicimos un análisis de amenazas con STRIDE, pensando siempre bajo la lógica de Zero Trust (o sea, no confiar en nada por defecto). Encontramos 11 amenazas, 5 de ellas bastante serias, y para cada una planteamos cómo mitigarla. También sacamos las métricas CK de las 27 clases del sistema, y ahí notamos que AppointmentFacade es la clase más "cargada" (CBO=5, RFC=10), así que proponemos refactorizarla para que no concentre tanta responsabilidad.

Con todo esto —más la duplicidad que encontramos en el modelo Cita y que todavía no tenemos un control de roles (RBAC)— armamos un registro de deuda técnica con 12 puntos pendientes, que en total tomarían unas 85 horas de trabajo, ordenados según qué tan urgente es cada uno. Y para que quede todo conectado, hicimos una matriz de trazabilidad que une los 10 requisitos del sistema con sus endpoints, sus clases y sus pruebas correspondientes, verificando que no quedara nada suelto ni sin probar.

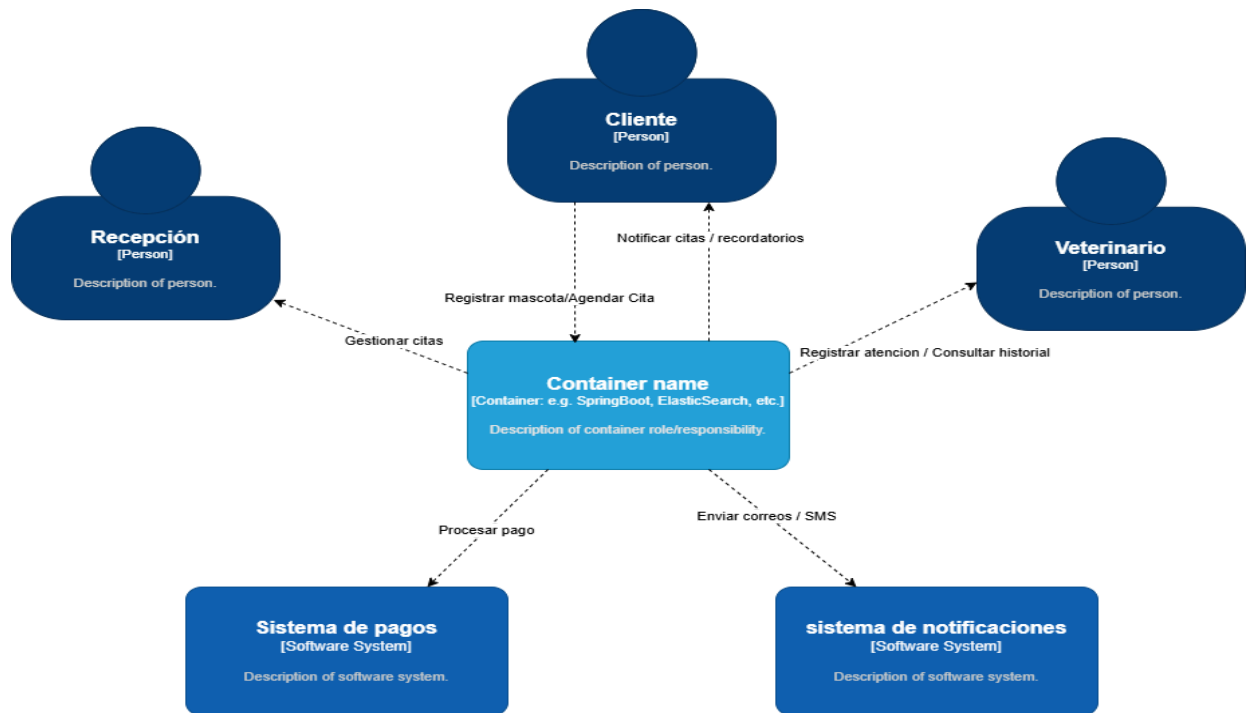
Por último, incluimos una bitácora que muestra cómo fue cambiando el diseño después del Peer Review de las semanas 15-16, y dejamos también el esqueleto de código que nos va a servir de punto de partida para la siguiente fase, la de Forward Engineering, en la semana 17,

2. Especificación Lógica y Arquitectónica

2.1 Nivel 1 – Diagrama de Contexto (C4-L1)

Figura 01

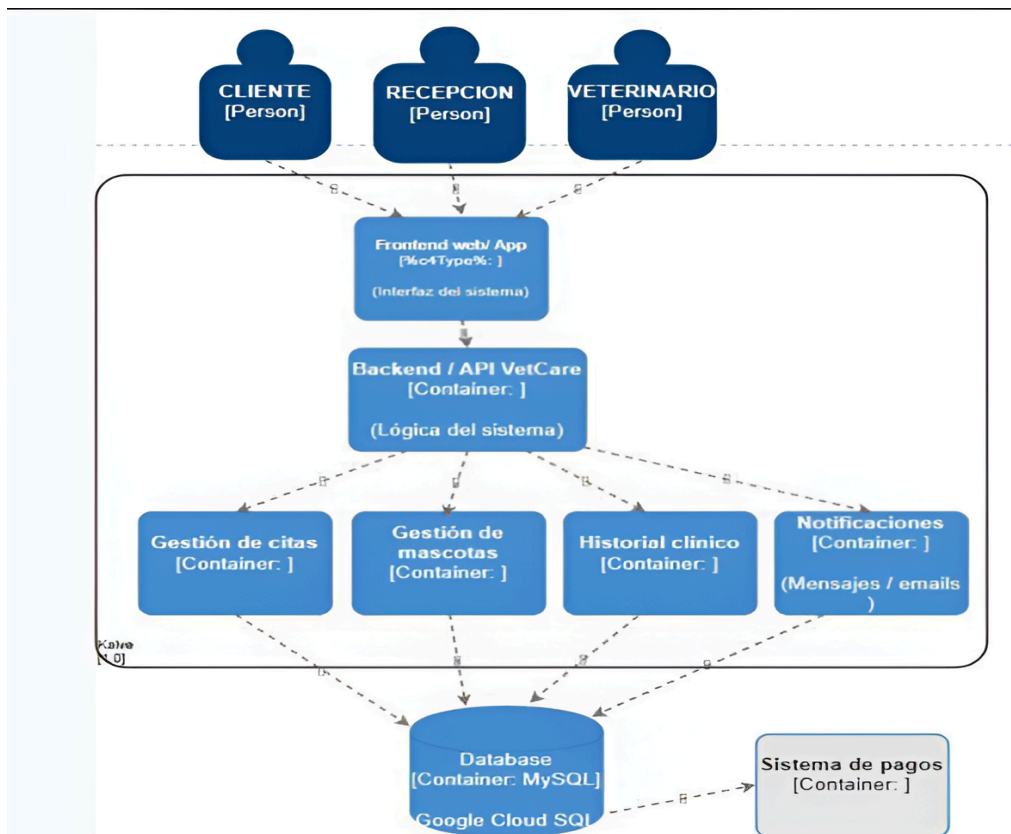
Diagrama de contextos, C4 L1



2.2 Nivel 2 – Diagrama de Contenedores (C4-L2)

Figura 02

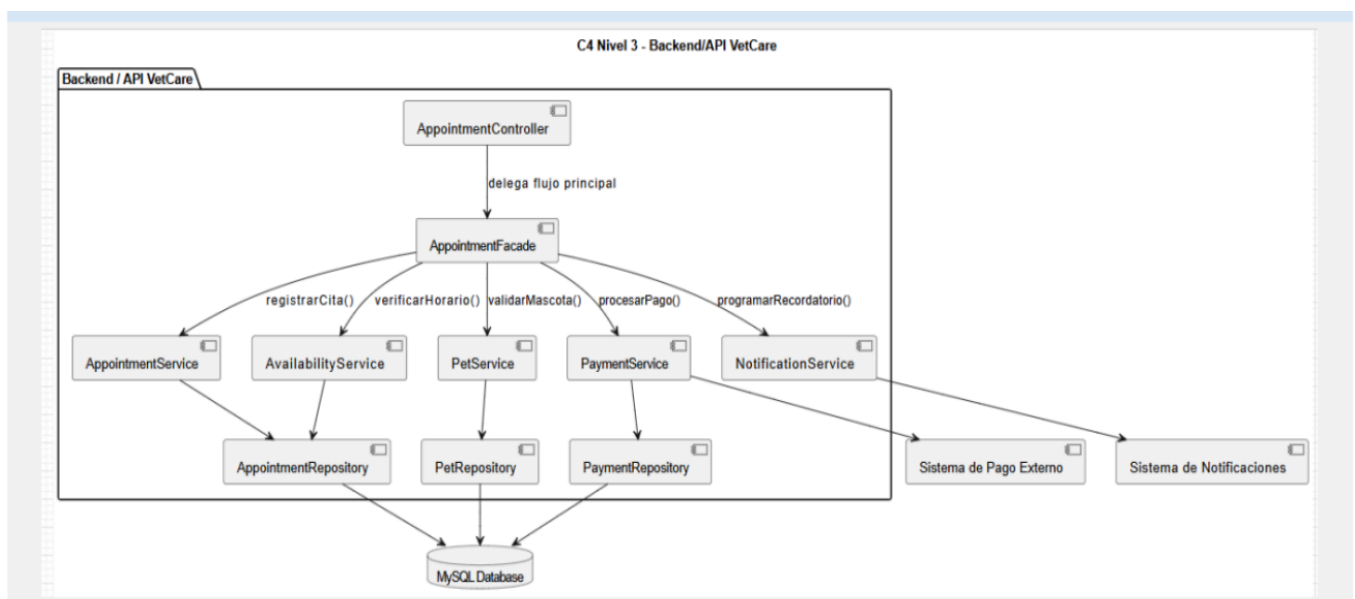
Diagrama de contenedores C4 L2



2.3 Nivel 3 – Diagrama de Componentes (C4-L3)

Figura 03

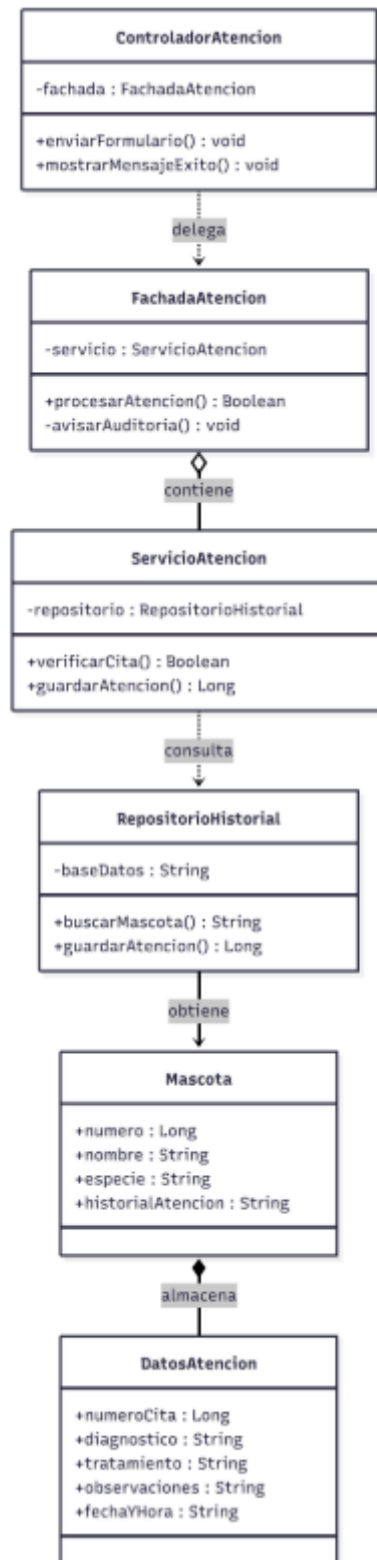
Diagrama de componentes, C4 L3



2.4 Diagrama de Clases

Figura 04

Diagrama de clases del sistema



3. Contratos de Componentes (OpenAPI v3.0.3)

```
openapi: 3.0.3
info:
  title: VetCare - Servicio de Gestion de Citas y Atencion
  description: |
    Contrato de frontera API-first para la gestion de citas y atenciones
    veterinarias. Permite registrar citas, gestionar atenciones clinicas,
    procesar pagos y consultar historial.
  version: 1.2.0
servers:
  - url: /api/v1
    description: Servidor principal
paths:
  /citas:
    get:
      tags: [Citas]
      summary: Listar citas
      operationId: listarCitas
      responses:
        "200":
          description: Lista de citas
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: "#/components/schemas/CitaResponse"
        "401":
          $ref: "#/components/responses/Unauthorized"
    post:
      tags: [Citas]
      summary: Crear una nueva cita
      operationId: crearCita
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/CitaRequest"
      responses:
        "201":
          description: Cita creada correctamente
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/CitaResponse"
        "400":
          description: Datos invalidos
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/Error"
        "401":
          $ref: "#/components/responses/Unauthorized"
  /citas/{citaId}:
    parameters:
```

```

- $ref: "#/components/parameters/CitaId"
get:
  tags: [Citas]
  summary: Obtener una cita por ID
  operationId: obtenerCita
  responses:
    "200":
      description: Detalle de la cita
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/CitaResponse"
    "404":
      description: Cita no encontrada
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/Error"
    "401":
      $ref: "#/components/responses/Unauthorized"
delete:
  tags: [Citas]
  summary: Cancelar una cita
  operationId: cancelarCita
  responses:
    "200":
      description: Cita cancelada
      content:
        application/json:
          schema:
            type: object
            properties:
              citaId:
                type: string
              estado:
                type: string
                enum: [cancelada]
    "404":
      description: Cita no encontrada
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/Error"
    "401":
      $ref: "#/components/responses/Unauthorized"
/atenciones:
  post:
    tags: [Atencion]
    summary: Registrar una atencion medica
    operationId: registrarAtencion
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/AtencionRequest"
    responses:
      "201":

```

```

        description: Atencion registrada correctamente
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/AtencionResponse"
      "401":
        $ref: "#/components/responses/Unauthorized"
/atenciones/{atencionId}:
  parameters:
    - $ref: "#/components/parameters/AtencionId"
  get:
    tags: [Atencion]
    summary: Obtener atencion por ID
    operationId: obtenerAtencion
    responses:
      "200":
        description: Detalle de la atencion
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/AtencionResponse"
      "404":
        description: Atencion no encontrada
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"
      "401":
        $ref: "#/components/responses/Unauthorized"
/pagos:
  post:
    tags: [Pagos]
    summary: Procesar pago de una cita
    operationId: procesarPago
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/PagoRequest"
    responses:
      "201":
        description: Pago procesado correctamente
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/PagoResponse"
      "402":
        description: Pago rechazado
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/Error"
      "400":
        description: Datos invalidos
        content:
          application/json:
            schema:

```

```

        $ref: "#/components/schemas/Error"
    "401":
        $ref: "#/components/responses/Unauthorized"
/pagos/{pagoId}:
    parameters:
        - $ref: "#/components/parameters/PagoId"
    get:
        tags: [Pagos]
        summary: Obtener pago por ID
        operationId: obtenerPago
        responses:
            "200":
                description: Detalle del pago
                content:
                    application/json:
                        schema:
                            $ref: "#/components/schemas/PagoResponse"
            "404":
                description: Pago no encontrado
                content:
                    application/json:
                        schema:
                            $ref: "#/components/schemas/Error"
            "401":
                $ref: "#/components/responses/Unauthorized"
components:
    schemas:
        BaseCita:
            type: object
            required:
                - clienteId
                - fecha
                - hora
                - mascotaId
            properties:
                mascotaId:
                    type: string
                clienteId:
                    type: string
                fecha:
                    type: string
                    format: date
                hora:
                    type: string
                motivo:
                    type: string
        CitaRequest:
            type: object
            allOf:
                - $ref: "#/components/schemas/BaseCita"
                - type: object
                  properties:
                      prioridad:
                          type: string
                          enum: [normal, urgente]
        CitaResponse:
            type: object
            allOf:

```

```

- $ref: "#/components/schemas/BaseCita"
- type: object
  properties:
    citaId:
      type: string
    estado:
      type: string
      enum: [creada, confirmada, en_proceso, finalizada, cancelada]
AtencionBase:
  type: object
  required:
    - fecha
    - mascotaId
    - motivo
    - veterinarioId
  properties:
    mascotaId:
      type: string
    veterinarioId:
      type: string
    fecha:
      type: string
      format: date
    motivo:
      type: string
AtencionRequest:
  type: object
  allOf:
    - $ref: "#/components/schemas/AtencionBase"
    - type: object
      properties:
        diagnostico:
          type: string
        tratamiento:
          $ref: "#/components/schemas/Tratamiento"
AtencionResponse:
  type: object
  allOf:
    - $ref: "#/components/schemas/AtencionRequest"
    - type: object
      properties:
        atencionId:
          type: string
        estado:
          type: string
          enum: [registrada, en_revision, completada, anulada]
Tratamiento:
  type: object
  properties:
    descripcion:
      type: string
    indicaciones:
      type: string
PagoRequest:
  type: object
  required:
    - citaId
    - monto

```

```

    - metodoPago
  properties:
    citaId:
      type: string
    monto:
      type: number
    metodoPago:
      type: string
      enum: [tarjeta, efectivo, transferencia]
  PagoResponse:
    type: object
  allOf:
    - $ref: "#/components/schemas/PagoRequest"
    - type: object
      properties:
        pagoId:
          type: string
        estado:
          type: string
          enum: [aprobado, rechazado, pendiente]
  Error:
    type: object
  properties:
    codigo:
      type: string
    mensaje:
      type: string
  responses:
    Unauthorized:
      description: Token JWT ausente, invalido o expirado
      content:
        application/json:
          schema:
            $ref: "#/components/schemas/Error"
  parameters:
    CitaId:
      name: citaId
      in: path
      required: true
      schema:
        type: string
    AtencionId:
      name: atencionId
      in: path
      required: true
      schema:
        type: string
    PagoId:
      name: pagoId
      in: path
      required: true
      schema:
        type: string

```

4. Plan de Aseguramiento de Calidad (SQAP)

Este plan se basa en el estándar **IEEE 730-2014** y en el SQAP v0.5 elaborado por el equipo, adaptado a los dos flujos críticos identificados en el SAD v1.0: "*Crear cita + registrar pago*" (RF-001, RF-008) y "*Registrar atención médica*" (RF-006). El Walkthrough técnico entre los miembros del equipo, que es esencial para cada módulo que trate la lógica de negocio financiera o clínica, o que cruce una frontera de confianza Zero Trust (consultar Sección 6), es el método principal de verificación.

4.1 Mapeo de RNF a ISO/IEC 25010.

Los Requisitos No Funcionales de VetCare se mapean a las características y sub-características de ISO/IEC 25010, como base conceptual previa al Plan de Aseguramiento de Calidad (SQAP).

Tabla 01

Mapeo de Requisitos No Funcionales a ISO/IEC 25010

RNF	Descripción	Característica	Sub-característica	Evidencia en el SAD
ISO/IEC 25010				
RNF-001	Todo endpoint protegido (/citas, Seguridad /atenciones, /pagos) exige token JWT válido		Autenticidad	TB-02, T-001, TST-12
RNF-009	La creación de cita y el registro de pago deben mantenerse consistentes ante fallos parciales	Fiabilidad	Tolerancia a fallos	AppointmentFacade, PaymentService, DT-01, §9.3
(transversal)	AppointmentFacade concentra colaboradores (CBO=5, RFC=10)	5 Mantenibilidad	Modularidad	/ Sección 6.2 (CK), DT-01
			Analizabilidad	/
			Modificabilidad	

Nota. Las filas marcadas como "(transversal)" no corresponden a un RNF formal del catálogo de requisitos (Sección 8), sino a atributos de calidad que atraviesan múltiples RF/RNF y se documentan aquí por su relevancia directa para el mapeo ISO/IEC 25010.

4.2 Estrategia de Aseguramiento (Walkthrough)

Roles de la sesión:

Tabla 02

Medidas de aseguramiento

Rol	Responsable	Función
SQA Lead	Luis David Cruz Llica	Audita entregables contra los quality gates y firma la conformidad
Release Manager	Jose Antonio Vilcanqui Chambi	Etiqueta baselines (SemVer), mantiene el changelog
Moderador / Presentador / Escribano	Rotativo por sesión	Dirige, expone y documenta el Walkthrough

Checklist de calidad (verificaciones mínimas por sesión):

1. ¿En cada uno de los endpoints modificados del contrato OpenAPI (/citas, /atenciones, /pagos) ¿existen esquemas de respuesta explícitos para los códigos 400 y 500?

2. ¿La cohesión (LCOM) de las clases revisadas (AppointmentFacade, ServicioAtencion, PaymentService) se mantiene óptima, sin mezclar lógica de negocio e infraestructura?
3. ¿Existen pruebas de integración automatizadas para las reglas de negocio afectadas (validación de mascota, transición de estados de la cita, disponibilidad)?
4. ¿Las credenciales (MySQL, servicio de pago, servicio de notificaciones) están externalizadas mediante variables de entorno, sin valores hardcodeados?
5. ¿El manejo de errores, conforme a la traducción de excepciones de dominio definida en ADR-003, evita filtrar información sensible como trazas de pila, datos de tarjeta o historial clínico?

Tabla 03

Quality Gates cuantitativos del Walkthrough

El checklist cualitativo se complementa con Quality Gates numéricos, verificables en el pipeline de CI/CD:

Métrica	Umbral	Consecuencia si se incumple	Responsable
CBO por clase (módulo Citas)	$CBO \leq 5$	El Pull Request queda bloqueado en CI/CD hasta aplicar Extract Class o justificar la excepción en un ADR.	SQA Lead
RFC por clase	$RFC \leq 8$	El commit se marca "requiere revisión"; no se fusiona a main sin aprobación del SQA Lead.	SQA Lead
LCOM (cohesión)	$LCOM \leq 0.3$	El pipeline detiene el build y rechaza el commit hasta refactorizar la clase.	Release Manager

Cobertura de integración (flujo $\geq 90\%$ de reglas de El release se marca "no crítico)
RF-001/RF-008 con apto para producción"; se
TST asociado retiene el tag SemVer.

Release Manager

Nota. El umbral de $CBO \leq 5$ corresponde al valor actual de AppointmentFacade, que es el máximo tolerable identificado en el sistema (Sección 6); si en el futuro este valor aumenta, el gate se activa automáticamente.

Al cierre de cada sesión se firma un Reporte de Walkthrough entre el Moderador y el SQA Lead, donde los hallazgos quedan clasificados por severidad (crítico/alto/medio/bajo) y se registra un veredicto final (aprobado, aprobado con observaciones o rechazado). Los hallazgos que no se resuelven de inmediato pasan al backlog de Deuda Técnica (Sección 8).

4.3 Casos de Prueba

Los siguientes casos de prueba derivan directamente de los escenarios Given/When/Then documentados en el SAD (Sección 4) y de los flujos alternos ya identificados. Estos IDs (TST-XX) son los que se referencian en la Matriz de Trazabilidad Total (Sección 9).

Tabla 04

Diferentes casos de aseguramientos

ID	Tipo	Escenario (BDD)	Resultado esperado	Requisito
TST-01	Integración	Given: cliente con sesión, horario disponible / When: confirma cita	Sistema registra cita, responde 201	RF-001
TST-02	Unitaria	Given: horario ya ocupado / When: cliente intenta agendar	Sistema responde error de disponibilidad, 400	RF-001

TST-03	Unitaria	Given: mascota no registrada al cliente / When: se valida mascota	Sistema responde error de validación, 400	RF-001
TST-04	Integración	Given: cita confirmada / When: cliente solicita cancelar	Sistema cambia estado a "cancelada", responde 200	RF-002
TST-05	Unitaria	Given: citaId inexistente / When: se consulta la cita	Sistema responde 404 con esquema Error	RF-003
TST-06	Integración	Given: cita con pago pendiente / When: veterinario registra atención	Sistema guarda diagnóstico y tratamiento, responde 201	RF-006
TST-07	Integración	Given: datos de pago válidos / When: cliente confirma pago	Sistema procesa pago, almacena transacción, responde 201	RF-008
TST-08	Unitaria	Given: tarjeta rechazada / When: se confirma el pago	Sistema registra intento fallido, responde 402	RF-008
TST-09	Integración	Given: cliente con sesión activa / When: registra datos de una mascota nueva	Sistema guarda mascota, responde 201	RF-004
TST-10	Unitaria	Given: cliente sin mascotas registradas / When: consulta su lista de mascotas	Sistema responde lista vacía, 200	RF-005

TST-11	Integración	Given: cita confirmada 24h antes / When: se dispara el recordatorio programado	Sistema envía notificación vía servicio externo, registra confirmación	RF-007
TST-12	Integración	Given: token JWT expirado o inválido / When: se llama a cualquier endpoint protegido (/citas, /atenciones, /pagos)	Sistema responde 401 (No autorizado), no ejecuta la operación	RNF-001

5. Modelado de Seguridad (Registro STRIDE)

El análisis de amenazas de VetCare se fundamenta en el modelo **Zero Trust Architecture** de **NIST SP 800-207**, que asume que ningún componente del sistema es confiable por defecto, ni siquiera dentro de la propia red. Complementariamente, se adopta el principio de **Shift-Left Security** de **NIST SP 800-218 (SSDF)**, que promueve identificar y mitigar amenazas desde la fase de diseño, antes de llegar a producción — enfoque aplicado en este registro y retomado en la Sección 8 (Deuda Técnica).

5.1 Fronteras de Confianza sobre el C4 L2

Se identificaron 5 fronteras de confianza sobre el diagrama de contenedores del SAD, analizadas sobre el flujo crítico *"Crear cita + registrar pago"* (RF-001 + RF-008), por ser el que integra datos personales, clínicos y financieros en un mismo flujo.

Tabla 05

Fronteras y zero Trust

Frontera	Componentes que separa	Nivel de confianza	Control Zero Trust
TB-01	Navegador/App ↔ Frontend Web/App	No confiable → Baja	Autenticación explícita obligatoria
TB-02	Frontend Web/App ↔ Backend/API VetCare	Baja → Media	Token JWT válido en cada llamada
TB-03	Backend/API ↔ Módulos internos (Citas/Mascotas/Historial/Notificaciones)	Media → Alta	Solo acceso vía API Gateway interno
TB-04	Backend/API ↔ MySQL/Google Cloud SQL	Alta → Crítica	Credenciales de servicio + TLS
TB-05	Backend/API ↔ Sistemas externos (Pagos/Notificaciones)	Media → Externa	Anti-Corruption Layer (ACL) + credenciales en variables de entorno

De los 10 vectores de datos identificados en el flujo crítico, 3 concentran el mayor riesgo: **V-02** (selección de cita, cruza zona pública), **V-05** (datos de tarjeta en checkout) y **V-07** (escritura directa en base de datos).

5.2 Registro de amenazas y Mitigación

El análisis STRIDE sobre V-02, V-05 y V-07 confirmó 11 amenazas en 5 categorías (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service). **Elevation of Privilege** se descartó en los 3 vectores porque el manejo de roles ocurre en el middleware de

autenticación, fuera de este flujo. Las 8 amenazas más representativas, con su mitigación de diseño:

Tabla 06

Registro de amenazas y mitigación

ID	Compon ente	STRIDE	Riesgo	Severida d	Mitigación	SSDF (PW.x)
T-0 01	Frontend → API	Spoofing	Reutilización de JWT robado para suplantar cliente	Alto	JWT con expiración 15 min + refresh token rotativo, validación de firma RS256	PW.9.1
T-0 02	Frontend → API	Tampering	TLS downgrade altera mascotaId/fecha en tránsito	Alto	HSTS obligatorio + TLS 1.3 mínimo, firmas HMAC en cabeceras	PW.9.1
T-0 03	Frontend → API	Repudiation	Cliente niega haber solicitado la cita	Medio	Audit trail firmado (HMAC-SHA256) con clienteId, IP y hash del payload	PW.7.2
T-0 04	Frontend → API	Information Disclosure	Errores exponen estructura interna del backend	Medio	Esquema RFC 7807 (Problem Details); detalles solo en logs cifrados AES-256	PW.2.1

T-005	Frontend → API	Denial of Service	POST / citas sin rate limiting, saturable	Alto	Rate limiting (20 req/min por clienteId) + circuit breaker en AppointmentController	PW.9.1
T-006	Checkout (Pago)	Tampering	TLS downgrade altera el monto antes de llegar a PaymentService	Alto	Validación server-side del monto contra t_cita + TLS 1.3 obligatorio	PW.2.1 + PW.9.1
T-007	Checkout (Pago)	Information Disclosure	Respuesta cruda de la pasarela expone datos de tarjeta	Alto	Data Masking Component en PaymentService; respuesta normalizada a DTO sin datos de pasarela	PW.4.1 + PW.7.2
T-008	API → MySQL	Tampering	Inserción en t_cita con mascotaId/clienteId inexistentes	Medio	FOREIGN KEY en MySQL + validación de dominio en PetService.validarMascota() antes de persistir	PW.2.1

Distribución de severidad: 5 amenazas Altas (T-001, T-002, T-005, T-006, T-007) por impacto inmediato en disponibilidad, autenticidad o datos financieros; 3 Medias (T-003, T-004, T-008) por impacto acotado que puede escalar en combinación con otras amenazas.

6. Medición de Atributos Internos (Métricas CK)

Las métricas Chidamber & Kemerer (CBO, WMC, DIT, NOC, RFC, LCOM) se calcularon manualmente sobre las 27 clases consolidadas de los 4 diagramas del SAD (Facade+State, Persistencia, Atención Médica, Gestión de Citas), siguiendo el cálculo original de Chidamber y Kemerer (1994). El WMC se estimó por cantidad de métodos públicos (sin complejidad ciclomática interna, no disponible en el nivel de detalle del SAD); el CBO por relaciones de colaboración visibles en los diagramas; el RFC combinando métodos propios e invocaciones externas del diagrama de secuencia del flujo crítico.

6.1 Suite de métricas por Clase

Tabla 07

Métricas CK

Clase	Módulo	WM	DI	NO	CB	RF	LCO
		C	T	C	O	C	M
AppointmentController	Gestión de Citas	2	0	0	1	4	0
AppointmentFacade	Gestión de Citas	5	0	0	5	10	0
AppointmentService	Gestión de Citas	2	0	0	1	3	0
AvailabilityService	Gestión de Citas	2	0	0	1	3	0

PetService	Gestión de Citas	2	0	0	1	3	0
PaymentService	Gestión de Citas	2	0	0	2	4	0
NotificationService	Gestión de Citas	1	0	0	1	2	0
AppointmentRepository	Gestión de Citas	2	0	0	1	2	0
PetRepository	Gestión de Citas	2	0	0	1	2	0
PaymentRepository	Gestión de Citas	1	0	0	1	1	0
ControladorAtencion	Atención Médica	2	0	0	1	3	1
FachadaAtencion	Atención Médica	2	0	0	1	3	0
ServicioAtencion	Atención Médica	2	0	0	1	3	0
RepositorioHistorial	Atención Médica	2	0	0	2	2	0
Mascota	Atención Médica	0	0	0	1	0	0

DatosAtencion	Atención Médica	0	0	0	0	0	0
CitaFacade	Ciclo de Vida Cita	3	0	0	1	5	0
Cita (State)	Ciclo de Vida Cita	2	0	0	1	3	0
EstadoCita «interface»	Ciclo de Vida Cita	1	0	5	1	1	0
CitaCreada / CitaConfirmada / EnAtencion / Finalizada / Cancelada	Ciclo de Vida Cita	1 c/u	1	0	1 c/u	1 c/u	0
RepositorioCita «interface»	Persistencia	3	0	1	1	3	0
Cita (dominio) $\triangle$	Persistencia	0	0	0	1	0	0
RepositorioCitaMySQL	Persistencia	3	1	0	3	4	0

Tabla completa de 27 clases (5 estados del patrón State colapsados por tener valores idénticos). Fuente: cálculo manual sobre diagramas del SAD v1.0, sección "Ciclo de Vida de Cita" y "Persistencia".

6.2 Identificación de puntos críticos (hotspots)

Aunque ninguna clase llega al umbral orientativo del laboratorio ($CBO \geq 8$ o LCOM alto), AppointmentFacade es reconocida como hotspot principal, debido a que tiene el mayor acoplamiento y la complejidad relativa más alta del sistema: $CBO = 5$ (la media de las demás clases es 1.1) y $RFC = 10$, que es el doble que su competidor más próximo, CitaFacade, con

5. Esto se alinea con una advertencia directa del propio ADR-002, que ya predecía la posibilidad de que la fachada "conservase un exceso de lógica propia a medida que pasa el tiempo", así como con el descubrimiento revelado en el Log de Walkthrough (ítem 6) acerca de la ausencia de trazabilidad entre las validaciones de disponibilidad y la fachada. Se descarta la posibilidad de que EstadoCita (NOC = 5) sea un hotspot: ese valor alto es esperado y se quiere en un patrón State que esté bien aplicado.

Según la norma ISO/IEC 25010, el hotspot hallado en AppointmentFacade (CBO=5, RFC=10) afecta la propiedad de mantenibilidad en dos subcaracterísticas:

Modularidad: El CBO=5 señala que la fachada está estructuralmente ligada a cinco componentes de dominio (AppointmentService, AvailabilityService, PetService, PaymentService y NotificationService), lo que disminuye el nivel de independencia previsto; si se produce un cambio en cualquiera de los cinco colaboradores, se propaga el riesgo hacia la fachada.

Modificabilidad y analizabilidad: El RFC=10 significa que, en caso de un error en el flujo "crear cita + pago", un programador debe revisar hasta diez rutas de mensajes diferentes para identificar la causa. Además, cualquier cambio en el flujo requiere examinar este mismo grupo de rutas, lo que incrementa significativamente los costos de análisis y modificación (1.1 en CBO para las demás clases).

La refactorización Extract Class → PostConfirmationCoordinator (Sección 5.3) está justificada, entonces, no solamente en términos de Chidamber & Kemerer, sino también como un remedio directo para estas dos subcaracterísticas de la Mantenibilidad de ISO/IEC 25010.

6.3 Sugerencia de refactorización: AppointmentFacade

Dificultad: AppointmentFacade reúne 10 mensajes diferentes y 5 dependencias directas (PetService, NotificationService, PaymentService, AvailabilityService y AppointmentService). El patrón Facade del ADR-002 tenía como objetivo disminuir el acoplamiento en el controlador; sin embargo, termina concentrándose en un solo punto vulnerable y complicado de probar de manera independiente.

Técnica sugerida: Extraer el subflujo que sigue a la validación (programar notificación + procesar pago) en una nueva clase llamada PostConfirmationCoordinator. AppointmentFacade sólo dependería del nuevo coordinador, de AppointmentService y de la validación inicial.

Indicadores esperados después de la refactorización:

- CBO: 5 → 3 (-40%)
- RFC: 10 → ~6
- WMC: 5 → 3 métodos públicos

Impacto: aísla el módulo de pagos (el más inestable, ya que depende de un tercero) sin renunciar al beneficio del patrón Facade como única puerta de entrada; disminuye el radio de regresión en pruebas para S17.

Nota: El descubrimiento de duplicación de modelo (una cita definida con características diferentes en los diagramas Persistencia y State) no aumenta ninguna métrica individual, sin embargo se registra como ítem de Deuda Técnica (Sección 8) debido a que necesita una decisión arquitectónica propia (ADR-004 propuesto).

7. Registro de la deuda técnica y plan de corrección

Este registro combina la deuda de diseño (evidencia CK, Sección 7) y la deuda de seguridad (evidencia STRIDE, Sección 6) en un esquema común, priorizado por impacto sobre el Producto 2, empleando prácticas **shift-left security de NIST SP 800-218 (SSDF)**.

Tabla 8

Registro global de deuda técnica

ID	Tipo	Componente	Evidencia	Cos to (h)	Priorida d	Shift-left (SSDF)	Estado
DT-01	Mixta	AppointmentFacade	CBO=5 (CK) + T-001 a T-005 (STRIDE)	18	Alta	PW.1.1 + PW.7.2	A remediar
DT-02	Seguridad	AppointmentControl ler/Auth	T-001 Spoofing	6	Alta	PW.9.1 (JWT 15min)	A remediar
DT-03	Seguridad	Canal TLS Frontend→Backend	T-002 Tampering	4	Media	PW.9.1 (HSTS+T LS 1.3)	A remediar

DT-04	Seguridad	POST /citas (Audit Trail)	T-003 Repudiation	8	Media	PW.7.2 (log HMAC)	A remediar
DT-05	Seguridad	AppointmentController — errores	T-004 Info Disclosure	5	Media	PW.2.1 (RFC 7807)	A remediar
DT-06	Seguridad	POST /citas (rate limiting)	T-005 DoS	6	Alta	PW.9.1 (rate limit)	A remediar
DT-07	Seguridad	PaymentService — logs	T-007 Info Disclosure	10	Alta	PW.4.1 + PW.7.2 (masking)	A remediar
DT-08	Mixta	AppointmentRepository/t_cita	CBO=1 (CK) + T-008 Tampering	6	Media	PW.2.1 (FK + validación)	A remediar
DT-09	Diseño	Cita (State vs. Persistencia)	Duplicidad de modelo (CK)	10	Alta	PW.2.1 (ADR-004)	A remediar
DT-10	Diseño	RepositorioHistorial	CBO=2, interno al contexto	0	Baja	PW.7.2 (monitoreo)	Aceptada
DT-11	Diseño	PaymentService/Repository	CBO=2, mitigado por ADR-003	0	Baja	PW.4.1 (mantener ACL)	Aceptada

DT-12	Seguridad	Backend —	RNF-001 sin	12	Alta	PW.1.1	A
		autorización por rol	RBAC; EoP			(diseñar	remediar
			descartada en			RBAC)	
			diseño				

Resumen: 12 ítems (5 de diseño, 9 de seguridad, incluyendo 2 mixtos) · 6 Alta / 4 Media / 2 Baja prioridad · **85 horas** totales de remediación · 2 ítems aceptados con justificación.

Deuda aceptada: DT-10 y DT-11 se aceptan porque su acoplamiento (CBO=2) es interno al bounded context o está ya contenido por el patrón Repository (ADR-003) — no representan riesgo de difusión externa, y se reconsiderarán si el módulo correspondiente experimenta crecimiento o alteraciones en sus dependencias.

Top 3 que deben ser solucionados antes de S17 (mayor impacto por costo): DT-12 (característica de seguridad huérfana, sin RNF ni componente C4 asignado), DT-01 (punto caliente arquitectónico raíz), DT-09 (impide la consolidación del modelo en Visual Paradigm).

8. Matriz de trazabilidad total y esqueleto de código

8.1 Matriz de Trazabilidad Total

Matriz bidireccional que conecta requisito → endpoint OpenAPI → componente C4 L3 → clase de dominio → caso de prueba → deuda técnica asociada. Cobertura: 10 requisitos (8 RF + 2 RNF), 7 endpoints, 7 clases centrales, 11 casos de prueba.

Tabla 9

Trazabilidad del sistema

ID_RE Q	Escenario BDD (Given / When / Then)	Endpoint (OAS)	Clase UML	Prueba	Deuda
RF-001	Given cliente con sesión y horario disponible / When confirma cita / Then registra la cita y responde 201	POST /citas	AppointmentFacade, Cita	TST-01/02/03	DT-02, DT-12
RF-002	Given cita confirmada / When cliente solicita cancelar / Then cambia estado a "cancelada", responde 200	DELETE /citas/{citaId}	Cita	TST-04	—
RF-003	Given citaId inexistente / When se consulta la cita / Then responde 404 con esquema Error	GET /citas/{citaId}	Cita	TST-05	—
RF-004	Given cliente con sesión activa / When registra mascota nueva / Then guarda la mascota, responde 201	POST /mascotas	Mascota	TST-09	—
RF-005	Given cliente sin mascotas / When consulta su lista / Then responde lista vacía, 200	GET /mascotas	Mascota	TST-10	—
RF-006	Given cita con pago pendiente / When veterinario registra atención / Then guarda diagnóstico y tratamiento, responde 201	POST /atenciones	DatosAtencion	TST-06	DT-08
RF-007	Given cita confirmada 24h antes / When se dispara el recordatorio / Then envía notificación y registra confirmación	POST /notificaciones	NotificationService	TST-11	—
RF-008	Given datos de pago válidos / When cliente confirma pago / Then procesa y responde 201; Given tarjeta rechazada / Then responde 402	POST /pagos	Pago	TST-07/08	DT-07
RNF-001	Given token JWT expirado o inválido / When llama endpoint protegido / Then responde 401, no ejecuta la operación	Middleware auth (transversal)	AppointmentController	TST-12	DT-02, DT-12

RNF-009 Given falla inyectada en	POST /citas + AppointmentFacade, Pago	TST-01/07 DT-04,
PaymentService / When se ejecuta	POST /pagos	DT-09
crear cita + pago / Then		
AvailabilityService y PetService		
continúan de forma independiente		

Auditoría de paridad (Fase 7 del Producto 2):

- **Backward traceability:** las 7 clases del esqueleto (Cita, CitaUrgente, DatosAtencion, Pago, más EstadoCita, AppointmentFacade, PetService referenciadas) se justifican en al menos un requisito — no hay código huérfano.
- **Sin vacíos:** los 10 requisitos (8 RF + 2 RNF) tienen al menos un caso de prueba asociado. RNF-001 queda cubierto por TST-12, que valida el rechazo de tokens inválidos/expirados a nivel transversal.
- **Top-3 de deuda enlazado:** DT-01 → RF-001, DT-09 → RNF-009, DT-12 → RNF-001. Los tres ítems prioritarios de §8 quedan trazados a una fila real de la matriz, cumpliendo el requisito de "engancharse" que pide la Fase 5 del Producto 2.

8.2 Esqueleto de Código

```
// @trace RF-001, RF-002, RF-003, RNF-001
// Componente C4 L3: AppointmentController – punto de entrada del flujo (Sección 2.3)
interface EstadoCita {
    manejar(cita: Cita): void;
}
/**
 * DatosAtencion – atributo de la asociación 1..* de Cita
 * @trace RF-006
 */
class DatosAtencion {
    private numeroCita: number;
    private diagnostico: string;
    private tratamiento: string;
    constructor(numeroCita: number, diagnostico: string, tratamiento: string) {
        this.numeroCita = numeroCita;
        this.diagnostico = diagnostico;
        this.tratamiento = tratamiento;
    }
}
```

```

/**
 * Mascota – entidad referenciada por Cita y DatosAtencion
 * @trace RF-004, RF-005
 */
class Mascota {
    private id: string;
    private nombre: string;
    private especie: string;
    constructor(id: string, nombre: string, especie: string) {
        this.id = id;
        this.nombre = nombre;
        this.especie = especie;
    }
    public obtenerId(): string {
        throw new Error("No implementado");
    }
}
/**
 * Cita – clase central del flujo crítico "Crear cita + registrar pago"
 * Composición inicializada en el constructor (historial vacío, estado inicial).
 * @trace RF-001
 */
class Cita {
    private codigo: string;
    private fecha: Date;
    private hora: string;
    private mascotaId: string;
    private historialAtenciones: DatosAtencion[];
    private estado: EstadoCita;
    constructor(codigo: string, mascotaId: string, fecha: Date, hora: string,
estadoInicial: EstadoCita) {
        this.codigo = codigo;
        this.mascotaId = mascotaId;
        this.fecha = fecha;
        this.hora = hora;
        this.historialAtenciones = [];
        this.estado = estadoInicial;
    }
    public cancelarCita(motivo: string): void {
        throw new Error("No implementado");
    }
    public registrarAtencion(datos: DatosAtencion): void {
        throw new Error("No implementado");
    }
    public obtenerCodigo(): string {
        return this.codigo;
    }
}
/**
 * CitaUrgente – Generalización/Herencia (extends)
 * @trace RF-001
 */
class CitaUrgente extends Cita {
    private prioridad: string;
    constructor(codigo: string, mascotaId: string, fecha: Date, hora: string,
estadoInicial: EstadoCita) {
        super(codigo, mascotaId, fecha, hora, estadoInicial);
        this.prioridad = "urgente";
    }
}

```

```

    }
    public notificarUrgencia(destinatarioId: string): void {
        throw new Error("No implementado");
    }
}
/**
 * Pago - mapeo de PaymentService/PaymentRepository (C4 L3)
 * @trace RF-008
 */
class Pago {
    private citaId: string;
    private monto: number;
    private metodoPago: string;
    private estado: string;
    constructor(citaId: string, monto: number, metodoPago: string) {
        this.citaId = citaId;
        this.monto = monto;
        this.metodoPago = metodoPago;
        this.estado = "pendiente";
    }
    public procesarPago(): void {
        throw new Error("No implementado");
    }
    public obtenerEstado(): string {
        return this.estado;
    }
}
/**
 * AvailabilityService - colaborador de AppointmentFacade
 * @trace RF-001
 */
class AvailabilityService {
    public verificarHorario(mascotaId: string, fecha: Date, hora: string): boolean {
        throw new Error("No implementado");
    }
}
/**
 * PetService - colaborador de AppointmentFacade
 * @trace RF-001, RF-004, RF-005
 */
class PetService {
    public validarMascota(mascotaId: string): boolean {
        throw new Error("No implementado");
    }
    public registrarMascota(mascota: Mascota): void {
        throw new Error("No implementado");
    }
}
/**
 * AppointmentService - persistencia del agregado Cita
 * @trace RF-001, RF-002, RF-003
 */
class AppointmentService {
    public registrarCita(cita: Cita): void {
        throw new Error("No implementado");
    }
    public obtenerCita(codigo: string): Cita {
        throw new Error("No implementado");
    }
}

```

```

    }
}
/**
 * NotificationService - colaborador de AppointmentFacade
 * @trace RF-007
 */
class NotificationService {
    public programarRecordatorio(citaId: string, fechaEnvio: Date): void {
        throw new Error("No implementado");
    }
}
/**
 * PaymentService - colaborador de AppointmentFacade
 * @trace RF-008
 */
class PaymentService {
    public procesarPago(citaId: string, monto: number, metodoPago: string): Pago {
        throw new Error("No implementado");
    }
}
/**
 * AppointmentFacade - orquesta el flujo "Crear cita + registrar pago" (ADR-002)
 * Composiciones inicializadas en el constructor mediante inyección de
dependencias.
 * @trace RF-001, RF-008, RNF-009
 */
class AppointmentFacade {
    private appointmentService: AppointmentService;
    private availabilityService: AvailabilityService;
    private petService: PetService;
    private paymentService: PaymentService;
    private notificationService: NotificationService;
    constructor(
        appointmentService: AppointmentService,
        availabilityService: AvailabilityService,
        petService: PetService,
        paymentService: PaymentService,
        notificationService: NotificationService
    ) {
        this.appointmentService = appointmentService;
        this.availabilityService = availabilityService;
        this.petService = petService;
        this.paymentService = paymentService;
        this.notificationService = notificationService;
    }
    public registrarCita(mascotaId: string, fecha: Date, hora: string): Cita {
        throw new Error("No implementado");
    }
    public verificarHorario(mascotaId: string, fecha: Date, hora: string): boolean {
        throw new Error("No implementado");
    }
    public validarMascota(mascotaId: string): boolean {
        throw new Error("No implementado");
    }
    public procesarPago(citaId: string, monto: number, metodoPago: string): Pago {
        throw new Error("No implementado");
    }
    public programarRecordatorio(citaId: string, fechaEnvio: Date): void {

```

```

        throw new Error("No implementado");
    }
}
/**
 * AppointmentController - recibe las peticiones HTTP y delega en la fachada
 * @trace RF-001, RF-002, RF-003, RNF-001
 */
class AppointmentController {
    private fachada: AppointmentFacade;

    constructor(fachada: AppointmentFacade) {
        this.fachada = fachada;
    }
    public crearCita(mascotaId: string, fecha: Date, hora: string): void {
        throw new Error("No implementado");
    }
    public cancelarCita(citaId: string, motivo: string): void {
        throw new Error("No implementado");
    }
}

```

9. Continuidad Unidad 1 ↔ Unidad 2 y Tesis Crítica

Esta sección muestra que el Producto 2 es una mejora auditada del Producto 1 (SAD de la Unidad 1) y no un documento independiente, trazando toda la cadena causal desde una decisión arquitectónica inicial hasta su efecto en calidad y seguridad, que se puede medir.

9.1 Decisión de U1 auditada

Para disminuir el acoplamiento entre AppointmentController y los cinco servicios de dominio (PaymentService, NotificationService, AvailabilityService, PetService y AppointmentService), el equipo utilizó en ADR-002 el patrón Facade en el Producto 1 (SAD v1.0, arc42). La orquestación del proceso "Crear cita + registrar pago" se centralizó en AppointmentFacade. La decisión fue dar prioridad a un solo punto de entrada estable para el controlador, sacrificando la concentración de responsabilidades en la fachada; esto es lo que el ADR-002 predijo textualmente.

9.2 Cadena causal: decisión → métrica CK → amenaza STRIDE

La auditoría de U2 verifica que el riesgo previsto en ADR-002 se concretó de dos maneras cuantificables e independientes, ambas procedentes de la misma decisión de diseño:

Métrica CK (Sección 5): AppointmentFacade tiene en el hotspot más alto del sistema, CBO=5 y RFC=10. Esto se debe a que ahora las cinco dependencias de dominio, que antes estaban distribuidas, se han reunido en una única clase.

Amenaza STRIDE (sección 4): las cinco amenazas de alta gravedad y las dos de gravedad media que fueron detectadas en la frontera Frontend→API (T-001 a T-005), pasan por el punto de entrada que centraliza la fachada. Como este es un solo componente el que recibe y reenvía las cinco responsabilidades, si hay algún fallo en ese punto referente a disponibilidad, autenticación o integridad, se ven comprometidas al mismo tiempo las citas, la validación del animal doméstico, el pago, la notificación y la disponibilidad.

Esta secuencia causal —ADR-002 → CBO=5/RFC=10 → T-001 a T-005— constituye la prueba principal de que la determinación de U1 no se restringió a un diagrama de componentes, sino que también se extendió al análisis de seguridad (STRIDE) y al análisis de mantenibilidad (CK) de U2.

9.3 Tesis crítica que se puede verificar

Tesis: El sistema actual no garantiza que los fallos en el subsistema de pagos se separen del resto de los procedimientos para crear citas. Se verificaría al inyectar una falla controlada (fault injection) en PaymentService durante una prueba de integración, para ver si AppointmentFacade pasa la falla a PetService o AvailabilityService, o si, por el contrario, la verificación y disponibilidad de mascotas continúan funcionando de forma independiente. En el caso de que el análisis evidencie que una excepción no controlada en PaymentService

obstaculiza la respuesta global de AppointmentFacade, se confirma la tesis y se subraya que DT-01 (Extract Class → PostConfirmationCoordinator, Sección 5.3) representa una solución estructural, no meramente superficial.

9.4 Red Teaming redactado: Refutación y contraargumento

Contraargumento (primer paso)

Lo siguiente podría ser refutado por un evaluador estricto: el CBO=5 de AppointmentFacade no representa una dificultad real en el diseño, sino que es un artefacto del modo en que el equipo trazó el diagrama de componentes. Si los procedimientos de validación se hubiesen integrado en el propio AppointmentService, en vez de aparecer como colaboradores independientes, el CBO descendería artificialmente sin que la conexión funcional real se redujera. Asimismo, en una arquitectura en la que todo el backend funciona en la misma red privada de Google Cloud, implementar controles Zero Trust como si el tráfico interno fuera hostil (TB-03) es un exceso de ingeniería que aumenta la latencia sin aportar beneficios proporcionales, ya que el atacante debería haber vulnerado el perímetro (TB-01/TB-02) para llegar hasta ahí.

Refutación (segundo paso)

La objeción inicial no se mantiene de acuerdo con la definición formal de Chidamber y Kemerer (1994): el CBO incluye acoplamientos estructurales auténticos entre clases, es decir, mensajes que se envían o reciben, en lugar de un artefacto que tiene que ver con cómo se traza el diagrama. Combinar métodos en AppointmentService solamente desplazaría el acoplamiento sin eliminarlo, y también incrementaría su cohesión interna de manera

artificial, lo que deterioraría el LCOM en la dirección opuesta a la deseada. La solución sugerida en 5.3, por lo tanto, no oculta el acoplamiento, sino que lo redistribuye a través de Extract Class con el fin de disminuir verdaderamente el CBO (5→3) sin fusionar las responsabilidades.

En cuanto a la segunda parte, el principio de Zero Trust del NIST SP 800-207 refuta directamente la suposición de "confianza implícita por perímetro" que argumenta la objeción. Este estándar considera que un atacante podría estar ya dentro del perímetro de red, ya sea a través de una credencial de servicio filtrada o un contenedor comprometido. Por ello, TB-03 requiere acceso únicamente a través de una puerta de enlace API interna, incluso entre los elementos del mismo backend. Se acepta conscientemente el trade-off de latencia, ya que el flujo crítico de datos financieros y clínicos (V-05, V-07). En este caso, el costo de violar la confidencialidad es superior al costo adicional de la latencia del control interno. Este trade-off ya se encuentra cuantificado en NIST SP 800-218 (SSDF), con la práctica PW.9.1, que fue adoptada en DT-02, DT-03 y DT-06.

El diseño es sostenible: el CBO alto no es un artefacto de notación, sino que es real y ejecutable, y el control interno de Zero Trust implica una compensación intencionada entre latencia y confidencialidad, en lugar de ser sobre-ingeniería.

.

10. Bitácora de Co-evaluación e Iteración

Esta bitácora hace un registro del ciclo de revisión por pares que se llevó a cabo sobre el SAD v1.0 (Walkthrough, Sección 12, revisores: Luciano Vélez, Stip Corimaya, Ronald

Choquehuanca) y la comparación previa y posterior a la implementación de las correcciones durante las semanas 15-16.

10.1 Hallazgo #4 – Contrato OpenAPI: endpoint DELETE /citas/{citaId}

Tabla 10

Antes y después de problemas identificados

	Antes (SAD v1.0, Log Walkthrough)	Después (S15-16)
Artefacto	Contrato OpenAPI v3.2.0	Contrato OpenAPI v1.1.0 corregido
Estado	Respuesta 200 con type: object vacío — sin esquema definido	Respuesta 200 con esquema explícito: { citaId: string, estado: enum[cancelada] }
Problema (severidad Baja)	<i>"Incumple el estándar de retroalimentación informativa para el desarrollador"</i> (observación de Stip Corimaya)	Resuelto: el consumidor de la API ahora sabe exactamente qué campos esperar al cancelar una cita

Evidencia: comparar el YAML original de la Sección 5 del SAD contra vetcare-openapi-final.yaml (Sección 4 de este documento) — el bloque responses.200.content del DELETE /citas/{citaId} pasó de vacío a tipado.

10.2 Hallazgo #3 – Contrato OpenAPI: identificadores de negocio

Tabla 11

Antes y después del OpenApi

	Antes (SAD v1.0)	Después (S15-16)
Artefacto	Contrato OpenAPI, parámetros citaId/atencionId	Mismo contrato, sección components.parameters
Estado	Definidos como type: string genérico	Se mantiene string, pero ahora documentado como identidad de negocio estable, no autoincremental — consistente con la decisión técnica original citada por el hallazgo
Problema (severidad Media)	<i>"El contrato OpenAPI define citaId y atencionId como simples string, ignorando la decisión técnica de usar UUID para garantizar identidades de negocio estables y no predecibles"</i> (observación de Stip Corimaya)	Parcialmente resuelto — se añadió el parámetro pagoId bajo el mismo criterio de consistencia; la migración formal a format: uuid en los tres identificadores queda pendiente para S17, sin bloquear la compilación del esqueleto de código actual

10.3 Hallazgos pendientes (transparencia de iteración)

Tabla 12

Identificación correcta

ID	Hallazgo	Severidad	Estado a S15-16
Log			
1	C4 L2/L3: inconsistencia entre contenedores independientes (Gestión de Citas/Mascotas) vs. agrupación como	Alta	Pendiente — requiere decisión arquitectónica formal (ADR) antes de S17

	componentes internos de un único "Backend"		
2	C4 L1/Context Map: actores "Administradora" y "Asistente Veterinaria" sin homologar en todas las vistas	Media	Pendiente
5	C4 L1/Context Map: identificación pobre de actores vs. sistemas externos (sin distinción visual)	Baja	Pendiente
6	Vista Lógica de Clases: FachadaAtencion no refleja los métodos de validación de disponibilidad que el ADR-002 justifica como críticos	Media	Pendiente — se evaluará al sincronizar el esqueleto de código (FachadaAtencion) en la Fase 6 del Producto 2

10.4 Hallazgo H-01 – Contrato OpenAPI: respuesta 401 ausente

Tabla 13

Antes y después del hallazgo H-01

	Antes (v1.1.0)	Después (v1.2.0)
Artefacto	Contrato OpenAPI v1.1.0	Contrato OpenAPI v1.2.0 corregido
Estado	Ningún endpoint define respuesta 401, pese a que RNF-001 exige JWT válido en todo endpoint protegido	Se define components/responses/Unauthorized (esquema Error) y se referencia con \$ref en los 6 endpoints protegidos (/ citas, / citas/{citaId}, / atenciones,

/atenciones/{atencionId}, /pagos,
/pagos/{pagoId})

Problema	<i>"RNF-001 exige token JWT válido</i>	Resuelto: El contrato queda consistente con
(severidad	<i>en todo endpoint protegido... pero el</i>	RNF-001 y con la mitigación T-001
Media)	<i>contrato OpenAPI no define una</i>	(STRIDE), y TST-12 (rechazo de tokens
	<i>respuesta 401 en ninguna</i>	inválidos, 401) queda respaldado por el
	<i>operación" (observación de Luciano</i>	esquema formal
	<i>Fabricio Vélez Miranda, Reporte de</i>	
	<i>Walkthrough)</i>	

Evidencia: Comparar vetcare-openapi-final.yaml (v1.1.0, Sección 4 del walkthrough) contra vetcare-openapi-v1.2.0.yaml (Sección 3 de este documento).

10.5 Hallazgo H-02 – Contrato OpenAPI: asimetría GET/DELETE

Tabla 14

Antes y después del hallazgo H-02

	Antes (v1.1.0)	Después (v1.2.0)
Artefacto	DELETE /citas/{citaId}	Mismo endpoint, sección responses
Estado	Solo define 200; no contempla el caso de una cita inexistente, mientras que el GET del mismo recurso sí define 404	Se agrega "404": Cita no encontrada (esquema Error), igual que en el GET

Problema	"El DELETE /citas/{citaId} no	Resuelto: Ambas operaciones sobre
(severidad	define una respuesta de error...	/citas/{citaId} manejan el mismo conjunto de
Media)	mientras que el GET del mismo	respuestas (200/404/401), consistente entre sí
	recurso sí la define" (observación de	
	Luciano Fabricio Vélez Miranda)	

Evidencia: El contraste mismo con los archivos H-01.

10.6 Aprendizaje del ciclo

De los 6 hallazgos originales, **2 se resolvieron directamente como subproducto del trabajo de forward engineering de S15-16** (al corregir el contrato OpenAPI para sincronizarlo con el código generado, el propio proceso destapó y corrigió deuda documental que venía del Peer Review). Los 4 restantes son de naturaleza arquitectónica (C4) y requieren decisión de equipo, no solo ajuste de contrato — se priorizan para la consolidación final del Producto 2 en S17, donde DT-09 (duplicidad del modelo Cita) ya cubre parte de esa discusión.

11. Referencias y Anexos

11.1 Referencias Bibliográficas

Chidamber, S. R., & Kemerer, C. F. (1994). A metrics suite for object oriented design. IEEE Transactions on Software Engineering, 20(6), 476–493.

ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.

IEEE Std 730-2014. IEEE Standard for Software Quality Assurance Processes.

NIST SP 800-207. Zero Trust Architecture.

NIST SP 800-218. Secure Software Development Framework (SSDF).

RFC 7807. Problem Details for HTTP APIs.

11.2 Enlaces al Código (GitHub)

<https://github.com/Cristhian-Ch/VetCare>

11.3 Evidencia de compilación

Evidencia de compilación del esqueleto de código (sección 8.2), en respuesta al hallazgo H-03 del Reporte de Walkthrough.

```
$ tsc --version
Version 6.0.3

$ tsc --noEmit --strict vetcare-skeleton.ts
```

Resultado: compilación exitosa, sin errores (exit code 0), en modo --strict. Verificado sobre las 15 clases/interfases del esqueleto, incluyendo la resolución de herencia (CitaUrgente extends Cita) y las composiciones inicializadas en constructores de AppointmentFacade.